

Trabajo Práctico 1: Aprendizaje por Refuerzos

Inteligencia Artificial y Neurociencias

Fecha de entrega: **hasta Miércoles 2/9 23:59hrs.**

1. Objetivo

El objetivo del trabajo práctico es entrenar un agente capaz de jugar al **Gold Dice RL** y obtener la mayor cantidad posible de puntos al finalizar la partida.

El agente deberá interactuar con el ambiente únicamente a través de observaciones y acciones:

```
obs = env.reset(seed=seed)

done = False
while not done:
    action = agent.act(obs, env)
    obs, reward, done, info = env.step(action)
```

2. Descripción del juego

Gold Dice RL es un juego estocástico de un jugador. Cada partida dura 30 turnos. En cada turno, el jugador tira dados, obtiene oro y decide cómo usarlo.

El oro puede utilizarse para convertirlo en puntos, comprar más dados, mejorar los dados, comprar escudos o guardar el mejor dado de la tirada para el próximo turno.

El problema central es aprender a decidir entre puntuar ahora, invertir en dados para producir más en el futuro o protegerse del riesgo de tormenta.

3. Reglas principales

En cada turno:

1. Se tiran los dados disponibles.
2. La suma de la tirada se agrega al oro.
3. El agente observa el estado actual.
4. El agente devuelve una única acción.
5. El ambiente procesa la acción elegida.
6. Puede ocurrir una tormenta.

7. Avanza el turno.

Las tormentas ocurren con probabilidad 0.15. Si el jugador tiene escudo, pierde un escudo. Si no tiene escudo, pierde aproximadamente la mitad del oro acumulado.

4. Observación

El ambiente devuelve una observación con información del estado actual del juego. En modo diccionario:

```
obs = {
    "turn": ...,
    "points": ...,
    "gold": ...,
    "shields": ...,
    "roll_max": ...,
    ...
}
```

Cada grupo puede usar esta observación directamente o construir su propia representación del estado. Esto es parte del problema a resolver.

5. Acciones

Las acciones disponibles son:

Código	Acción	Descripción
0	PASS	No hacer nada (ahorrar).
1	SCORE	Convertir la cantidad de oro deseada en puntos.
2	BUY_DICE	Comprar un dado extra.
3	UPGRADE	Aumentar en +1 el bonus global aplicado a todos los dados.
4	BUY_SHIELD	Comprar un escudo.
5	STORE_BEST_DIE	Guardar el mejor dado de la tirada actual.

El ambiente provee métodos para consultar acciones válidas:

```
env.get_valid_actions(); env.get_action_mask();
```

6. Dados, mejoras, costos y límites

Los dados utilizados son dados comunes de seis caras:

$$\text{DICE_FACES} = [1, 2, 3, 4, 5, 6]$$

Los costos principales del juego son:

Acción	Costo	Descripción
BUY_DICE	$18 + 8 \cdot (\text{num_dice} - 1)$	Comprar un dado adicional, <code>num_dice</code> en +1.
UPGRADE	$8 + 8 \cdot \text{dice_bonus}$	Aumentar <code>dice_bonus</code> en +1.
BUY_SHIELD	5	Comprar un escudo.
STORE_BEST_DIE	4	Guardar el mejor dado de la tirada actual para el próximo turno.

No hay un límite explícito de cantidad de dados, mejoras o escudos.

7. Archivos provistos

Archivo	Descripción
<code>config.py</code>	Parámetros del juego: costos, horizonte, dados, tormentas.
<code>env.py</code>	Ambiente del juego.
<code>agents.py</code>	Baselines y agentes propios.
<code>renderer.py</code>	Visualizaciones simples.
<code>evaluate_agents.py</code>	Evaluación de agentes en múltiples partidas.
<code>run_example.py</code>	Ejemplo de uso del ambiente.

Se puede agregar o modificar código de agentes, scripts de entrenamiento, funciones auxiliares, representaciones de estado, recompensas de entrenamiento y métodos de aprendizaje.

No se debe modificar la lógica interna de *env.py* ni de *config.py*.

8. Entrega y evaluación

El trabajo puede realizarse en grupos de 2 a 4 integrantes. Para aprobar, cada grupo deberá entregar:

- Un agente entrenado con alguna técnica vista en clase, por ejemplo Monte Carlo, SARSA, Q-Learning Tabular, DQL u otra variante justificada.
- Un agente ya entrenado que pueda jugar partidas completas sin intervención manual y ser evaluado de forma similar a las heurísticas baseline para el torneo.
- El código utilizado para entrenar el agente.
- Un informe de hasta 3 páginas explicando: estado, acciones, recompensa, algoritmo, hiperparámetros, resultados obtenidos, comparación contra baselines y qué funcionó mejor o peor durante el trabajo.

Al finalizar el TP se realizará un torneo entre agentes. Cada grupo debe elegir el que considere mejor. El torneo se realizará sobre múltiples partidas con distintas semillas aleatorias, utilizando como métrica principal el puntaje final promedio. Antes de la entrega habrá un **leaderboard en vivo** (pedir permisos) con `evaluate_agents.py` (`n_episodes=1000`, `seed=0`), mientras que el torneo final utilizará semillas privadas para evitar sobreajuste.